Method Selection and Planning

Cohort 1 Group 7

Group Members:

Kyle Clifton (wnp512)

Bailey Eyers (rkh552)

Matt Hollyhead (fpk506)

William Martin (zjc524)

Max Pither (rkm538)

Enkhtuguldur Sarantsetseg (wbj512)

Ronny Watt (dvn513)

Software Engineering Methods and Tools

Our team adopted the Agile Scrum methodology to develop the 2D maze game. Agile was chosen because it allows rapid iteration, continuous feedback, and flexibility in handling evolving requirements — which fits the dynamic nature of a small student project. With Agile, we divided the 6-week development cycle into short sprints focused on incremental deliverables. As the project progressed, we started to focus on tasks individually so that each task had someone working on it and could be completed by the deadline.

Tools Used:

- **GitHub**: For version control, issue tracking, and collaboration. It enabled every member to push updates, review commits, and track progress transparently.
- **Visual Studio Code**: Used as our primary IDEs for coding the game logic, integrating graphics, and testing builds. All members of our team are familiar with this IDE, so it is the best choice to program our code in.
- **Whatsapp**: Used for daily communication, quick decision-making, and sharing updates.
- **LibGDX**: Used as our chosen game engine for developing 2D maze game. We chose this because: it supports multiple platforms as well as having web support; it has a well proven and reliable framework; and it has a very active community to allow for support with development.
- **PlantUML**: Used for architecture and diagrams. Used as it supports the creation of a wide array of diagrams, that we will use to design our program. It is also quite simple as it uses an intuitive language to allow users to generate diagrams.
- **Google Drive**: Used to share documents and keep them together in one place where we can all work on them. Accessible to all group members using our university google accounts.

Justification:
When deciding which tool we wanted to use for each purpose, we researched multiple tools and considered the different pros and cons of each. The tools we chose fit our needs the best and ensured that the project could be worked on efficiently.

Alternative consideration:
Other IDEs we could have used include Eclipse and IntelliJ, but as everyone has more experience with VS Code, none of us decided to use these other IDEs.
We researched other game engines such as JMonkey and Unreal before deciding on LibGDX. We decided against JMonkey as it is only used for 3D games and our game is 2D, so JMonkey was unfit for our project. Unreal engine had a similar issue, but was also a lot more complex to use and is often used to create larger, more high-end games than the 2D maze game we are developing.
An alternative approach to architecture could have been to manually draw UML diagrams using other software such as LucidChart or Draw.io, but this could be tedious due to having to drag in, and then write in the contents of, each element in each diagram.

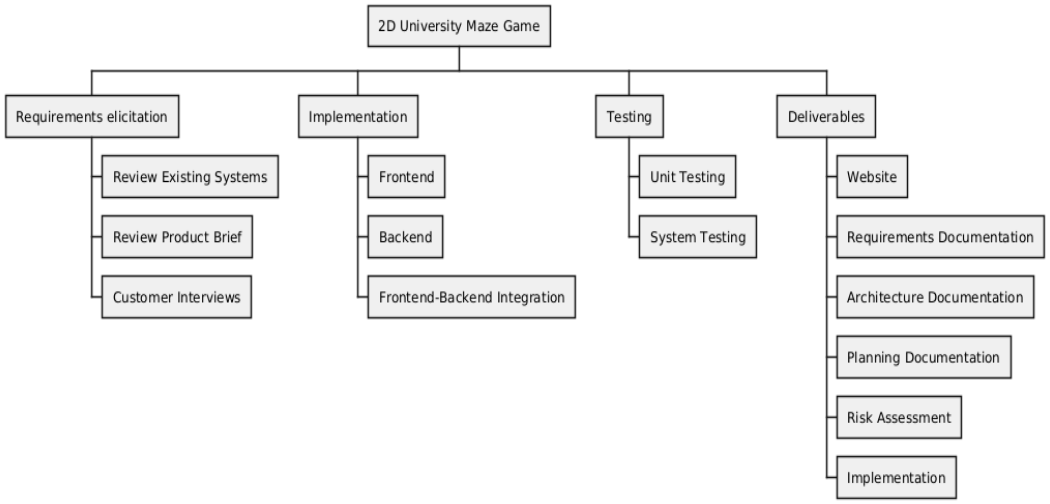
Team Approach:

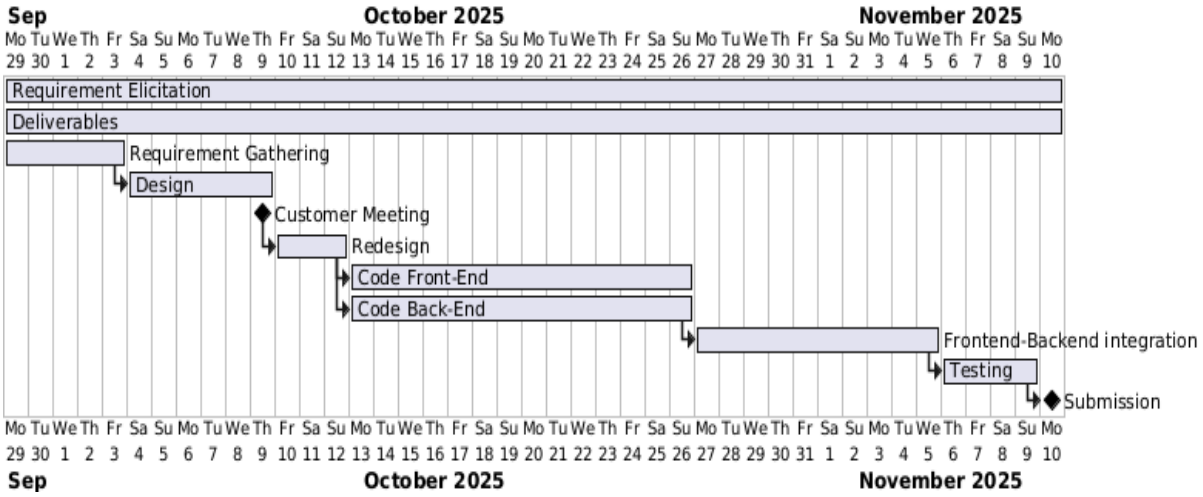
Our team approach was to split up our team into different tasks that needed to be completed for the project, i.e. architecture, implementation, risk assessment, .etc. Most tasks had 2 people working on them so that collaboration could occur while each task was being completed, without any tasks being overcrowded. This ensured that everyone was working on a task and all tasks were being worked on. We initially discussed the base requirements given in the product brief in order to plan our game, distribute tasks and come up with questions to ask in our customer meeting. After the customer meeting, we focussed on our specific tasks as we now had the full list of requirements. Each week we met up to discuss where we were at in the project and to adjust our plan for the remainder of the project.

Justification:
As this is a small student project, we needed to schedule our time on this project around our differing availability. Therefore, working on the tasks individually in our own time and checking each other's tasks when we meet up has allowed us to ensure that each part of the project gets completed. Adjusting our plan each week ensured that we had a realistic goal for when everything would be completed and that any tasks that we were behind on could be prioritised.

Systematic Project Plan:

Our plan for the project stayed mostly the same, but slight changes were made each week depending on the stage that we were at for each task.





Requirements elicitation (WP1)
Requirement elicitation was conducted to match the customer's needs and determine the requirements for the 2D university-based maze game.
T1.1: Review existing systems (29/9/2025 to 9/10/2025) ● To build a 2D maze game, exploring the existing systems was necessary to build a game. Furthermore, reviewing existing systems with similar goals and approaches will provide optimal product approaches and help to build better games for customers. T1.2: Customer interview (29/9/2025 to 9/10/2025) ● Customer interview is done to clarify the customer's requirements. ● Dependencies: T1.1

Design (WP2)
We needed to design the methods and systems to make the game.
T2.1: UI Design (4/10/2025 to 26/10/2025) ● Design User Interface that is easy to navigate. ● Dependencies: T2.2 T2.2: System Design (4/10/2025 to 26/10/2025) ● Design system methods and classes. ● Dependencies: T1.1, T1.2 T2.3: Storyline (4/10/2025 to 26/10/2025) ● Design storyline that is interesting to follow throughout the game. ● Dependencies: T1.2, T1.1, T2.1

Implementation (WP3)

Following the previous package, code was created according to the design.

T3.1: Backend implementation (13/10/2025 to 26/10/2025)
- Coding the backend (logic)
- Dependencies: T2.2

T3.2: Frontend implementation (13/10/2025 to 26/10/2025)
- Coding the frontend (UI)
- Dependencies: T2.1, T2.2

T3.3: Frontend-Backend integration (27/10/2025 to 5/11/2025)
- Coding how the frontend and backend will interact.
- Dependencies: T2.2, T3.1, T3.2

Testing (WP4)

To ensure the code was usable, testing was performed to confirm its functionality.

T4.1: Unit tests (5/11/2025 to 9/11/2025)
- Testing individual methods of the project.
- Dependencies: T3.3

T4.2: System tests (5/11/2025 to 9/11/2025)
- Testing the entire project.
- Dependencies: T3.3, T4.1

Deliverables

All deliverables are due on 10/11/2025. The website and zip file that contains all other deliverables are submitted due on time.

D1: Website
- Website that contains deliverables, and public face of the project.

D2: Requirements documentation
- Goes over user and system requirements and how requirements were elicited and negotiated.

D3: Architecture documentation
- Goes over the design process of the architecture and gives visual diagrams of the architecture of the product.

D4: Method selection and planning documentation
- Contains team approach, software engineer method that is used and general project plan

D5: Risk assessment and mitigation documentation
- Goes over risk identification, risk analysis, risk mitigation and risk monitoring.

D6: Implementation documentation
- Justification of the libraries and licenses we used for the project.